

Infografía

Módulo 8 — Enjambres • Sesión 2

Flow fields: mover multitudes

UPB • Gestión I/2026

Plan de la clase

1. **El problema** — boids no tienen meta; A^* no escala a cientos.
2. **Campo de costo** — BFS desde la meta (escena 05).
3. **Campo de flujo** — cada celda apunta a la meta (escena 06).
4. **Seguir el campo** — agentes baratos.
5. **Multitud** — flujo + separación (escena 07).

Boids (sesión 1) no van a ningún lado. Hoy: que cientos lleguen a una meta, baratísimo.

El problema

Boids se mueven juntos, pero **sin meta**: no van a ningún lado en particular.

Pathfinding (5.6) da el camino a una meta... pero un **A*** **por agente** no escala a cientos.

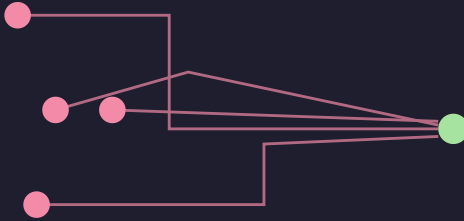
Si **todos** van a la **misma** meta, ¿hace falta buscar el camino una y otra vez, por cada agente?

*No. Resolvé el mapa **una** vez.*

Por qué flow fields

¿Por qué flow fields para multitudes?

Un A* por agente



N agentes $\rightarrow$ N búsquedas

caro: no escala a cientos

misma meta para todos $\rightarrow$ resolvé el mapa una vez, no por agente

Un solo flow field



1 búsqueda + N lecturas

barato: cientos casi gratis

La receta: dos campos sobre la grilla

1 · campo de costo

BFS desde la meta: pasos hasta ella

4	3	2	1
3	2	1	0
4	3	2	1

meta = 0

gradiente

2 · campo de flujo

cada celda → vecino más barato

↖	↖	↖	↓
→	→	→	●
↗	↗	↗	↑

se calcula UNA vez para todo el mapa – después cada agente solo lee su flecha

Paso 1 — campo de costo (escena 05)

Un **BFS** desde la meta: 0 en la meta, +1 por cada paso, las paredes no se cruzan.

```
costo.fill(INF)
var cola := [meta]; costo[_i(meta)] = 0.0
while not cola.is_empty():
    var c = cola.pop_front()
    for v in _vecinos4(c):
        if not paredes.has(v) and costo[_i(v)] == INF:
            costo[_i(v)] = costo[_i(c)] + 1.0
            cola.append(v)
```

Es la grilla de la sesión 5.6, pero en vez de un camino, la distancia a la meta DESDE TODOS lados.

Paso 2 – campo de flujo (escena 06)

Cada celda guarda **un vector**: hacia el vecino de **menor costo** (el que acerca a la meta).

```
var mejor := c
for v in _vecinos8(c):
    if costo[_i(v)] < costo[_i(mejor)]:
        mejor = v
flujo[_i(c)] = Vector2(mejor - c).normalized()
```

🎓 Tu turno: completá `_construir_flujo()`. Sin esto, los agentes no se mueven.

Seguir el campo: agentes baratos

El agente no piensa ni busca: lee el vector de su celda y camina.

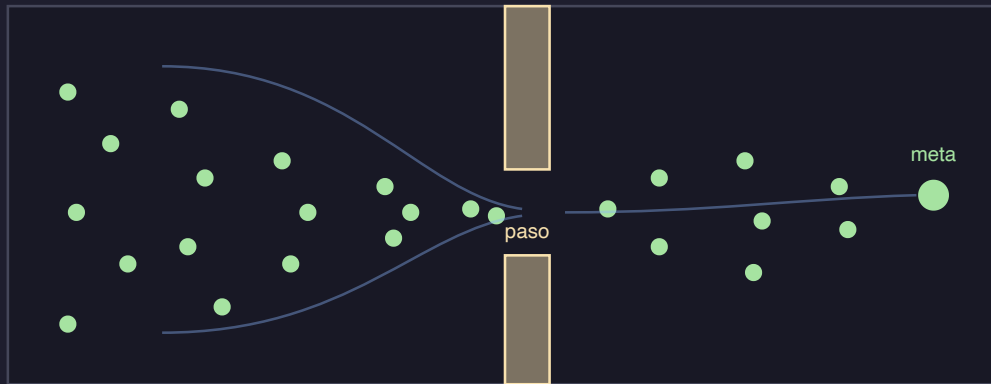
```
var deseada: Vector2 = campo.flujo_en(position) * MAX_SPEED  
velocity = velocity.move_toward(deseada, ACCEL * delta)  
position += velocity * delta
```

El trabajo difícil (el BFS) se hizo UNA vez. Sumar agentes es casi gratis.

Multitud: flujo + separación (escena 07)

200 agentes leyendo el mismo campo... **se enciman**. La cura es la **separación** de los boids (sesión 1).

Flow field + separación = navegación de multitudes



el flujo los lleva a la meta · la separación evita que se encimen en el paso

El cierre del módulo

```
var deseada := campo.flujo_en(position) * MAX_SPEED    # a DÓNDE ir  
# + separación de los boids:  
deseada += sep.normalized() * MAX_SPEED * peso_sep    # no ENCIMARSE
```

 Lo completamos en vivo: sumar la separación a la multitud.

Flow field (a dónde) + separación (no encimarse) = navegación de multitudes.

Resumen

Campo de costo: BFS desde la meta (distancia desde todos lados).

Campo de flujo: cada celda → vecino más barato.

Se calcula **una** vez; los agentes solo **leen** su celda.

+ separación = multitud que fluye sin encimarse.

Boids + flow fields: reglas locales simples que mueven a cientos. Eso es un enjambre.